QCon
全球软件开发大会

InfoQ 技术大会

~~欢迎来到自进化的时代~~

自进化智能体与基于记忆系统的强化学习

汇报人: 温睦宁

上海交通大学 人工智能学院

2026年10月20日



AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

目录

1 AI 落地的“最后一公里”

2 基于记忆的智能体自进化

3 垂域冷启动：NPU算子生成

一个模型会写代码，不代表它懂你的工程现场

通用模型综合能力很强，但垂域场景千千万，落到具体现场总感觉差那么一点。现场里最值钱的东西往往不在文档里：哪些约束容易踩坑、哪些修复真的有效、哪些经验下次还能复用。这些 Know-how 通用模型可能并不天生具备。

通用模型
它知道很多公开知识，也能做复杂推理。

→

垂域现场
真正的问题藏在工具链、数据、硬件和团队经验里。

→

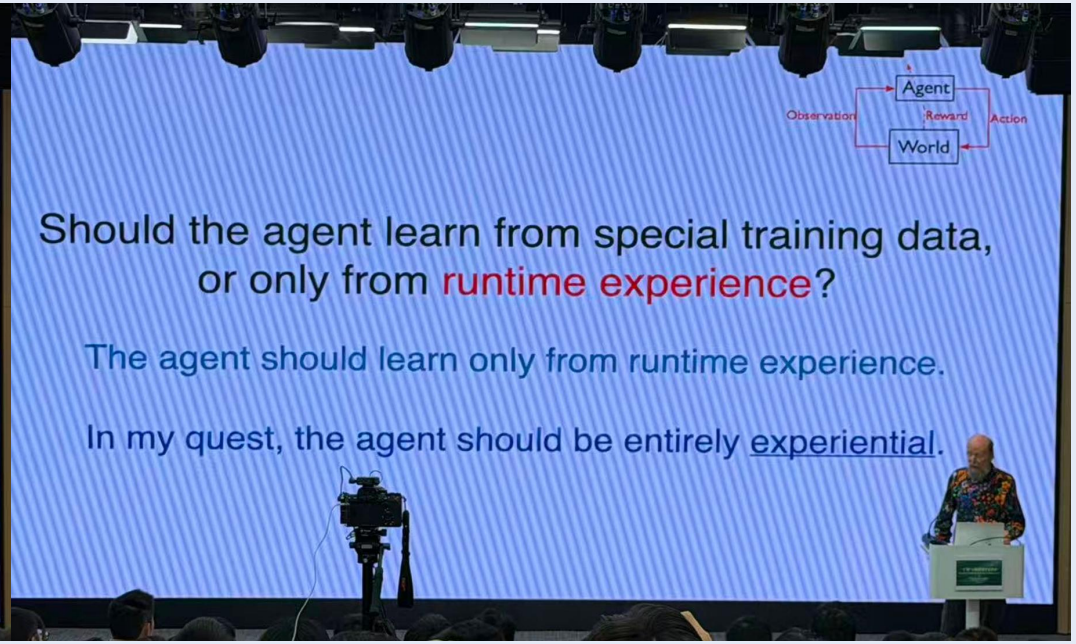
试错与迭代
一次次试错如果不沉淀，下一轮还会从头摸索。

缺的不是一次性答案，而是“每次运行后都能变聪明”的机制：记录经验、评估价值、在下一次任务里可控复用。

从运行时经验中学习

当Agent能够“Learn from Runtime Experience”这意味着人工智能范式从“静态的知识库”转变为“动态的适应者” 即，AI不仅在当前任务中学习，还能将这些经验转化为长期记忆，供未来使用。





自进化智能体的通用架构

大脑负责想，资产层负责记，反馈负责打分，更新器负责把“这次发生了什么”变成“下次优先怎么做”。不管结构多复杂，自进化智能体通常都能拆成这四个角色。



只要反馈不回写，Agent 再会行动，也很难叫“自进化”。

通用公式：自进化智能体 = 基座能力 + 可更新组件 + 反馈信号 + 更新规则

ICML 2026 中的自进化智能体

6,628

ICML 2026 官方接受论文

100% 全量标准筛查

613

弱相关关键词候选

占全库论文 9.25%

46

严格相关论文

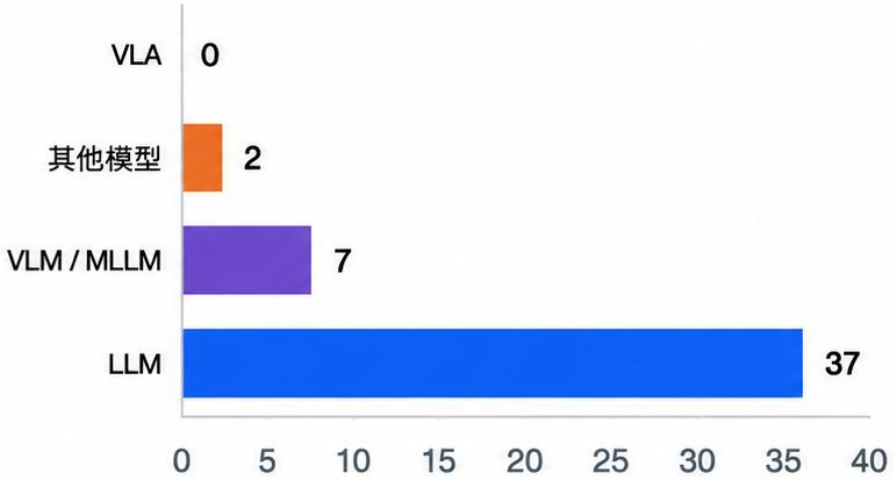
占全库论文 0.69%

严格相关集合

记忆、策略和技能/工作流三条主线合计 42 篇，评测与安全 4 篇。

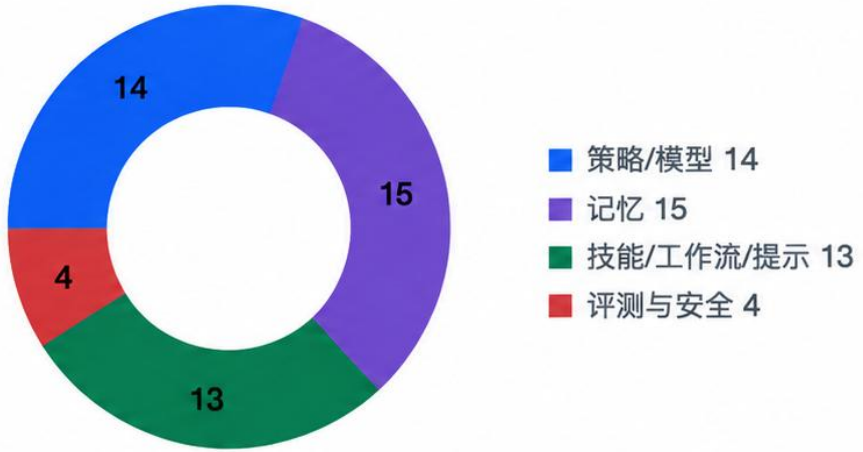
严格相关论文的模型基座

LLM 为绝对主流，未发现以 VLA 为主要基座的方法



严格相关论文的机制分类

记忆进化 15 篇最多，四类总计 46 篇



Agent0-VL：把求解与验证合成一个自进化回路

ICML 2026 • SELF-EVOLVING AGENTS

ORAL

Tool-Integrated Vision-Language Reasoning • 视觉语言推理

为什么重要

工具不再只是“查答案”，而是进入推理、过程验证和自修复；系统可在没有外部奖励的条件下形成内生改进闭环。

01 统一 Solver + Verifier

同一智能体提出解法、检查过程，并针对失败位置修复。

02 效果

相对基础模型平均 +12.5%；测试时扩展平均再 +7.3%。

官方全集		严格相关		重点解读
6,628	→	46	→	6

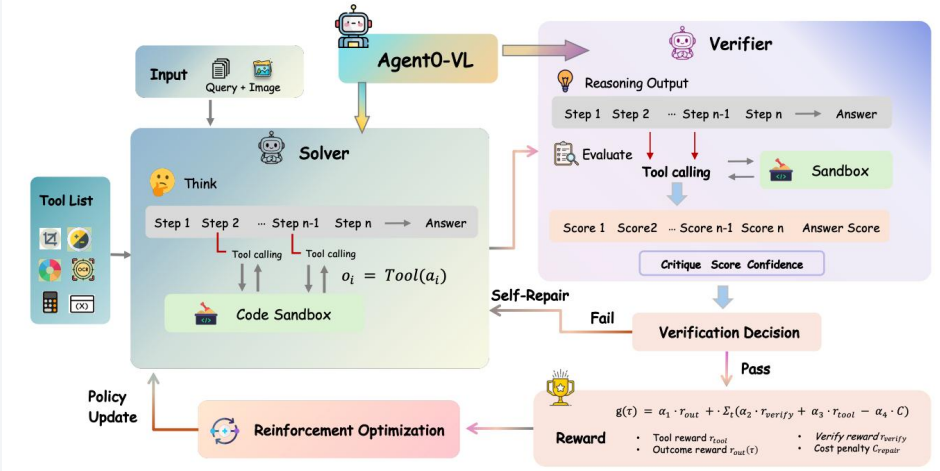


Figure 2. **The Framework of Agent0-VL.** The unified policy π_θ alternates between two internal roles: the **Solver** that generates reasoning trajectories with tool calls, and the **Verifier** that performs generative verification using tool feedback to produce critiques and step-wise rewards. These roles are jointly optimized through the Self-Evolving Reasoning Cycle, where self-generated rewards guide policy updates via RL.

3.1. Unified Solver-Verifier Architecture

To achieve autonomous evolution, as shown in Figure 2, Agent0-VL adopts a unified dual-role design, where a single LVLM alternates between two internal modes: a *Solver* that performs tool-integrated reasoning, and a *Verifier* that introspectively evaluates, critiques, and repairs the Solver's triplet (s_t, a_t, o_t), the Verifier outputs a feedback tuple:

$$V_t = (\text{score}_t, \text{conf}_t, \text{critique}_t),$$

where $\text{score}_t \in [-1, 1]$ measures factual correctness, $\text{conf}_t \in [0, 1]$ estimates epistemic certainty, and critique_t provides natural-language reflection describing potential rea-

Source: arxiv.org/abs/2511.19900 • ICML 2026 Oral

JitRL：冻结模型，也能在部署时做强化学习

POLICY EVOLUTION • NO GRADIENT

SPOTLIGHT

Just-In-Time Reinforcement Learning • Continual Learning Without Gradient Updates

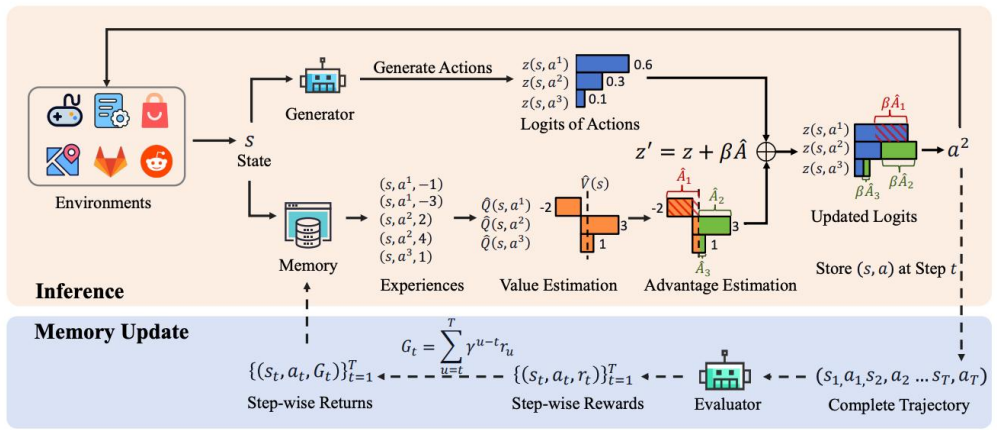


Figure 2. **Overview of the Just-In-Time Reinforcement Learning (JitRL) framework.** The system operates in a continuous loop: (1) In the *Inference* (top), the agent retrieves relevant past experiences $\mathcal{N}(s)$ from the non-parametric memory $\mathcal{M}$. The base LLM's logits z are then adjusted in closed-form ($z' = z + \beta\hat{A}$) using the estimated advantage A derived from historical returns, enabling test-time policy improvement without gradient updates. (2) In the *Memory Update* (bottom), completed trajectories are analyzed by an evaluator to compute discounted returns G_t. These new experiences are stored back into $\mathcal{M}$, allowing the agent to evolve its policy across episodes.

illustrated in Figure 2, the framework consists of three key components: (1) constructing a granular experience mem-

where $\gamma \in [0, 1]$ balances the weight of immediate versus future rewards.

01 检索经验

非参数记忆保存历史状态、动作与回报；当前任务到来时即时检索。

02 估计优势

根据相似轨迹在线估计 action advantage，不反向传播、不改权重。

03 直接改 logits

加性更新是 KL 约束策略优化的闭式精确解。

>30×

货币成本降低；且超过昂贵微调方法 WebRL

Source: arxiv.org/abs/2601.18510 • ICML 2026 Spotlight

QCon
全球软件开发大会

InfoQ 技术大会

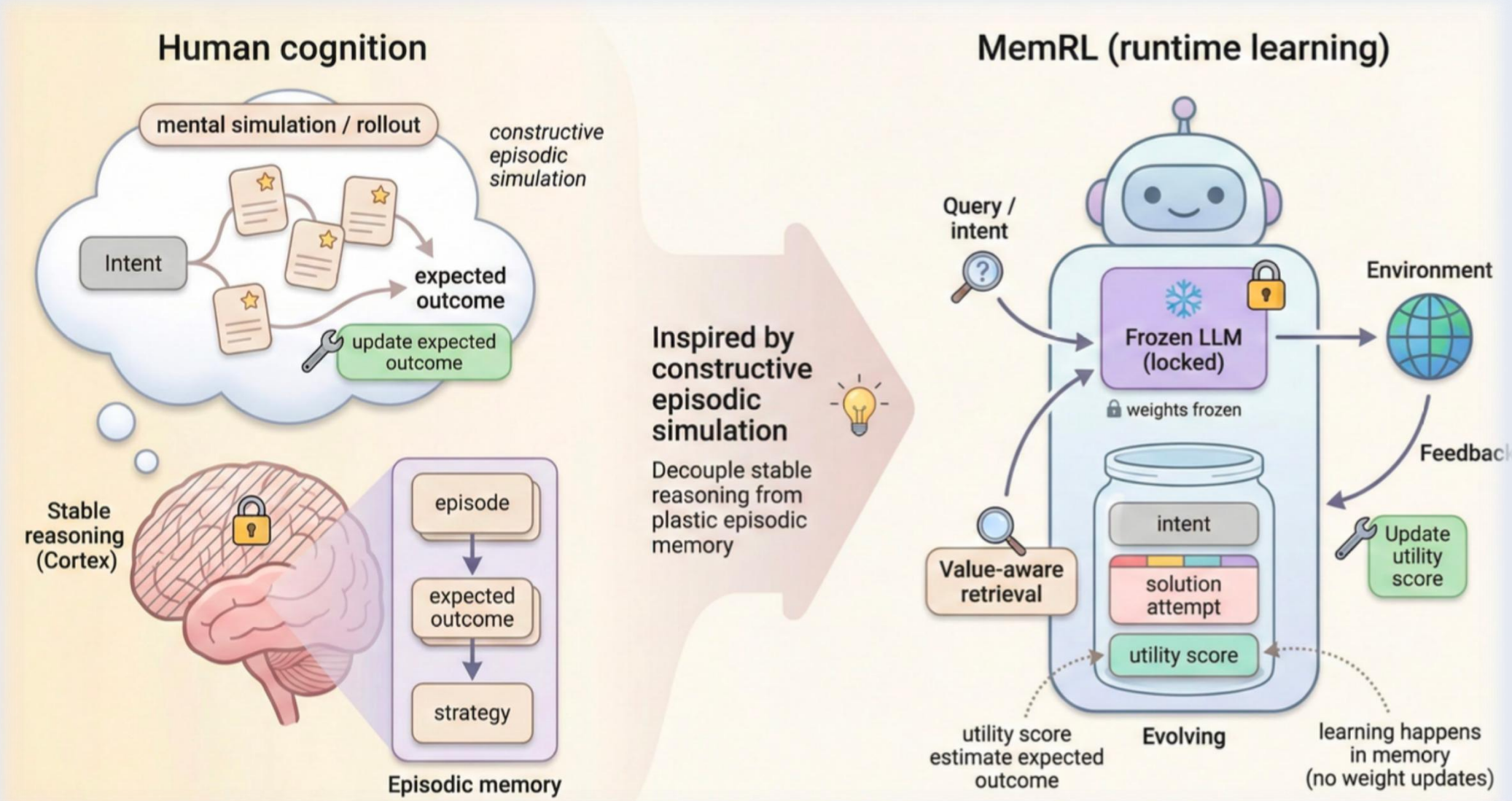
目录

1 AI 落地的“最后一公里”

2 基于记忆的智能体自进化

3 垂域冷启动：NPU算子生成

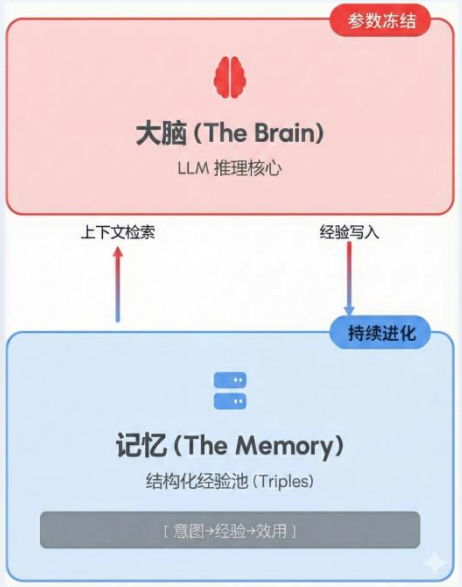
强化学习驱动的自进化记忆系统——MemRL



MemRL：智能体架构

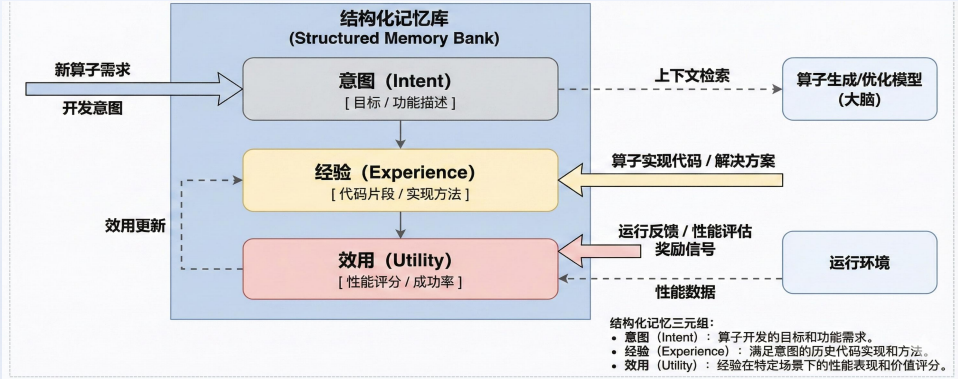
MemRL：结合记忆库，通过强化学习对记忆库中的经验“去粗取精”，进而赋予智能体“Learn from Runtime Experience”的能力。

“大脑-记忆”双核解耦架构



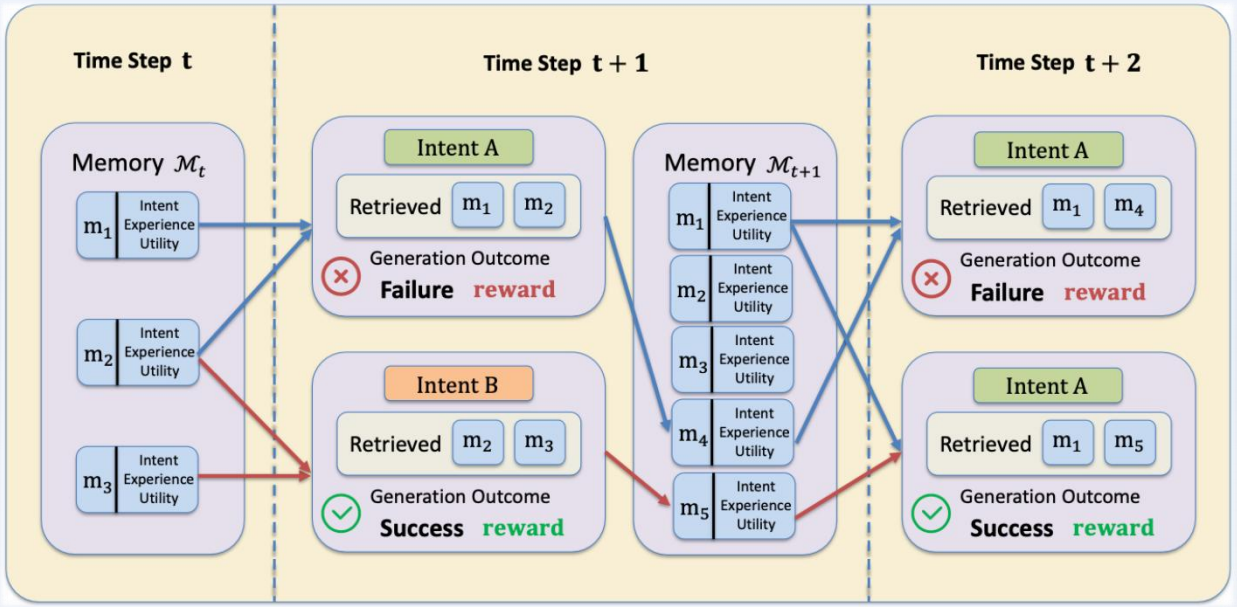
- 1、从“一次性生成”向“持续性进化”的范式改变。
- 2、大脑保持稳定：负责代码理解与逻辑推理，参数冻结，无需重训。
- 3、记忆持续更新：外挂结构化经验池。随环境反馈不断更新记忆内容与效用权重。

“意图-经验-效用”结构化记忆库



- 1、意图(Intent)：任务目标或用户需求。
- 2、经验(Experience)：满足意图的历史行为和轨迹。
- 3、效用(Utility)：在特定场景下参考历史经验的预期收益。

MemRL：作用于记忆的非参数化强化学习



1、智能体策略:

$$\pi(y_t|s_t,\mathcal{M}_t)=\sum_{m\in\mathcal{M}_t}\underbrace{\mu(m|s_t,\mathcal{M}_t)}_{\text{Retrieval Policy}}\cdot\underbrace{p_{\text{LLM}}(y_t|s_t,m)}_{\text{Inference Policy}} \quad (1)$$

2、最优检索策略:

$$\mu^*(m|s,\mathcal{M})=\arg\max_{m\in\mathcal{M}}Q(s,m) \quad (2)$$

3、MC Backup（示例）：

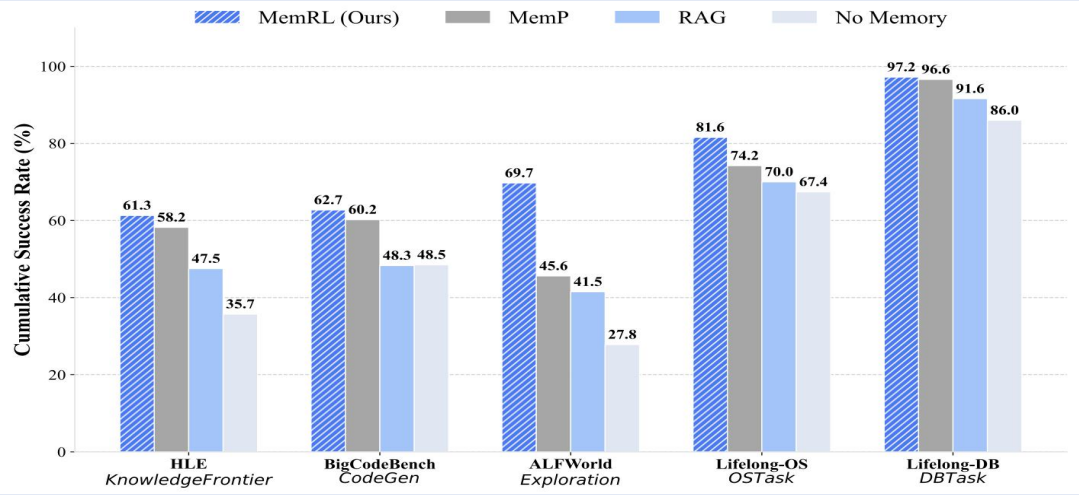
$$Q(s,m)\leftarrow Q(s,m)+\alpha\cdot(r_t-Q(s,m)) \quad (3)$$

认知科学出发的低成本自进化。将学习过程的目标从更新“大脑”参数转变为更新“记忆”与其中的“效用”值，使智能体的能力进化发生在廉价的记忆读写而非昂贵的模型重训上，同时将强化学习的理论保证带入智能体的 Runtime Learning过程中。

MemRL：效果与分析

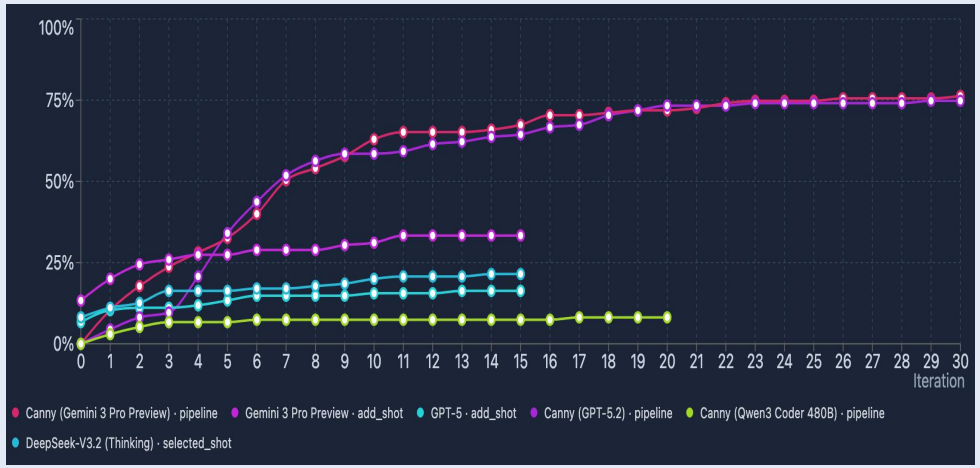
通用任务表现

- 在HLE (Humanity’ s Last Exam)上已突破60%+成功率。
- 当前的纯模型SOTA为37.2%（Gemini 3 Pro），集成Google Search后（Deep Research版本），智能体SOTA为45.8%；
- 还有Code、DB、OS、Math、Navigation、用户偏好跟随等场景。



国产 GPU 算子生成 （CANN）

- 核心难点在于预训练知识匮乏，考验模型的硬核智能；
- 昇腾平台算子生成正确率 80%+；
- 今年7月时，这方面的公开 SOTA 仅为 2.5%。
- L2 包括Attention相关复杂融合算子，如Flash Attn。



算子详细结果 https://rl4cann.zhuo.li/

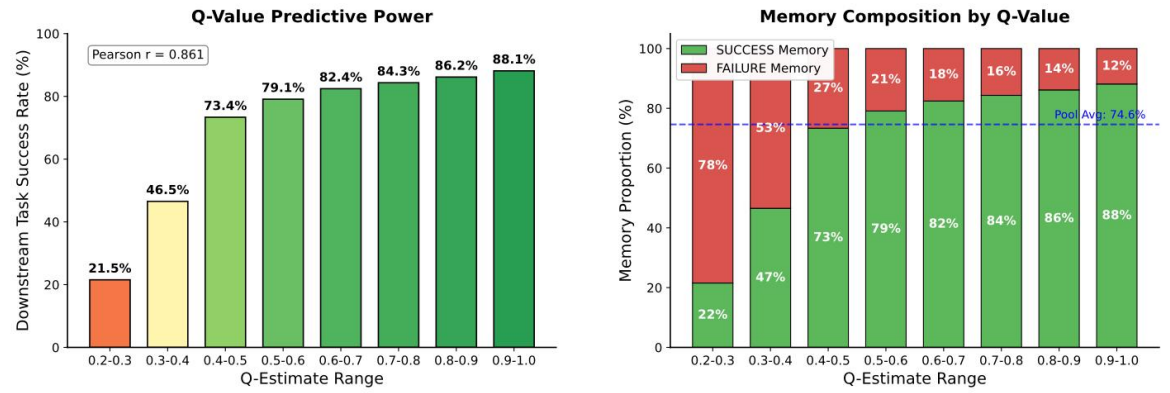
MemRL：分析实验

Q值分布分析

- 经过学习后，Q值与下游任务成功率成正相关。
- 高Q值记忆中依旧存在一批失败经验——有些失败也是高价值的。

稳定性观察

- 纯靠的Agentic pipeline也能在前期观察到可观的提升。
- 缺乏优化的理论保障，其上升很快会停止。



(a) Success Rate vs. Q-Range　　(b) Memory Composition

Figure 8: **Q-Value Analysis.** (a) Pearson $r = 0.861$ confirms Critic's predictive power. (b) Failure memories ($\sim$ 12%) in high Q-bins indicate latent strategic utility.

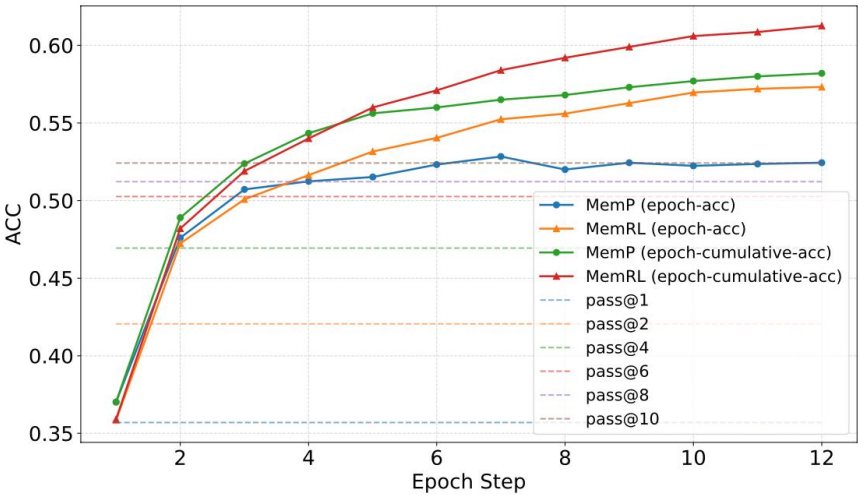
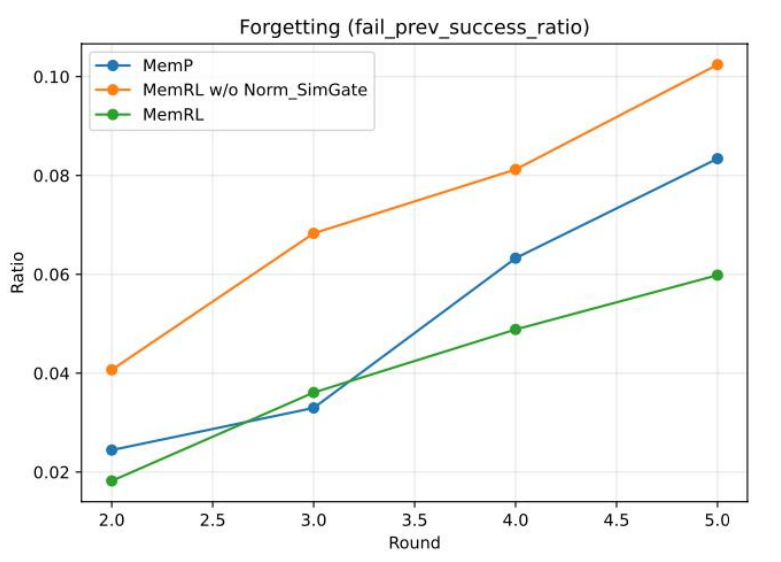


Figure 9: Epoch Acc and Cumulative ACC of MemRL and MemP in HLE.

MemRL：分析实验

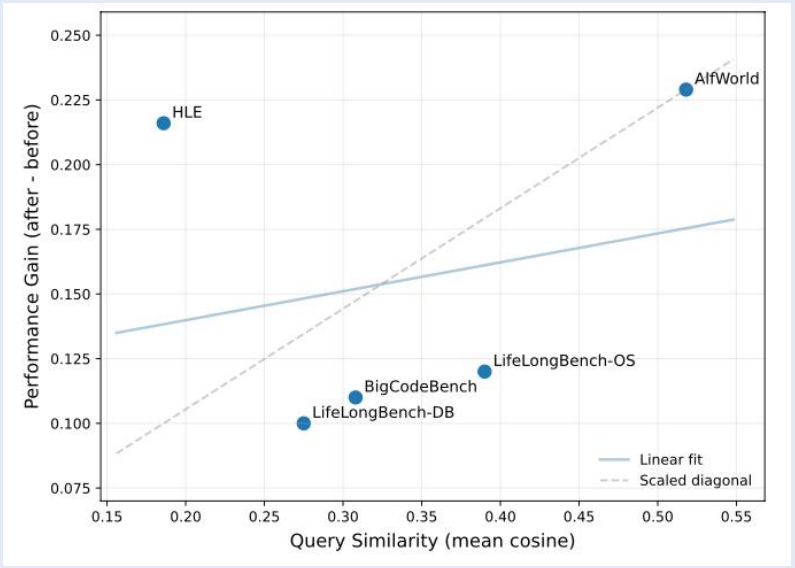
遗忘分析

- 遗忘现象相对减轻了，不过还是在上升——仍有优化空间。
- Normalization还是很重要的——传统RL似乎一直如此。



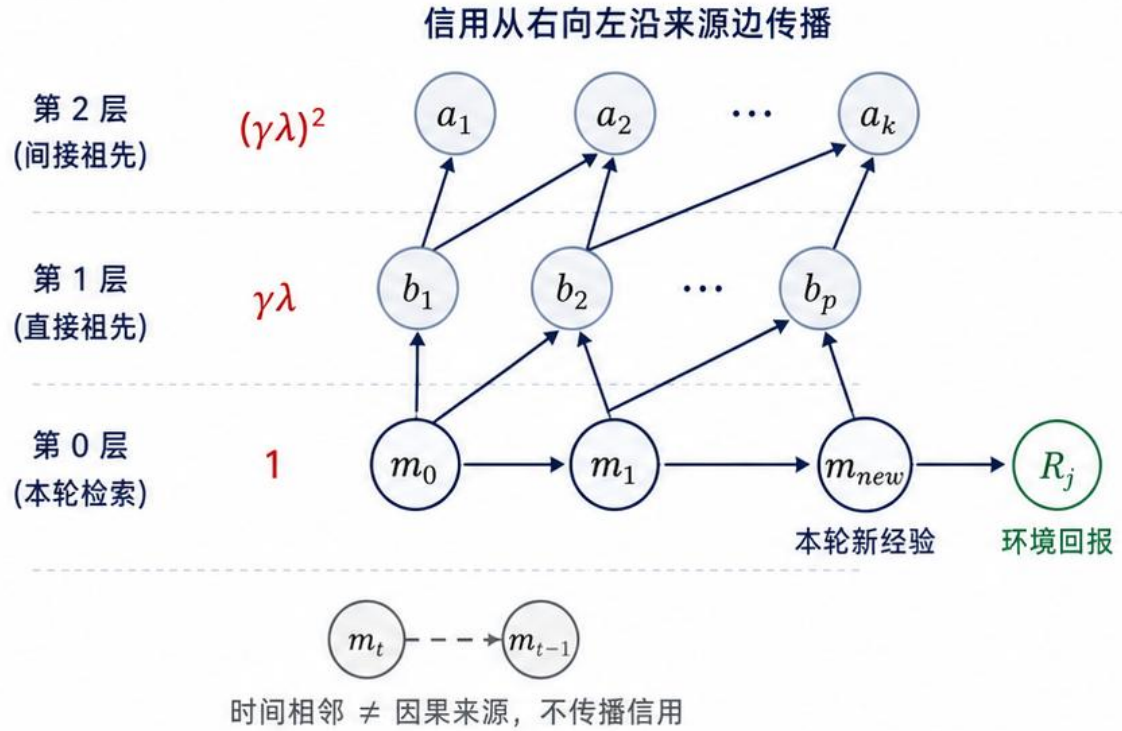
任务相似度观察

- 任务间相似度越高，泛化效果越好。
- HLE虽然任务多，但相似度还是太低了。
- 未来需要尽可能让训练任务足够多样与密集。



https://arxiv.org/abs/2601.03192

MemQ：用经验溯源图实现跨任务、多步结构归因



1 新经验初始化

$$Q(m_{new}, j) = |A_j|^{-1} \sum_{m_i \in A_j} Q(m_i)$$

2 直接检索经验的 TD 误差

$$\delta(m_i, j) = R_j + \gamma Q(m_{new}, j) - Q(m_i)$$

3 沿来源图传播结构信用

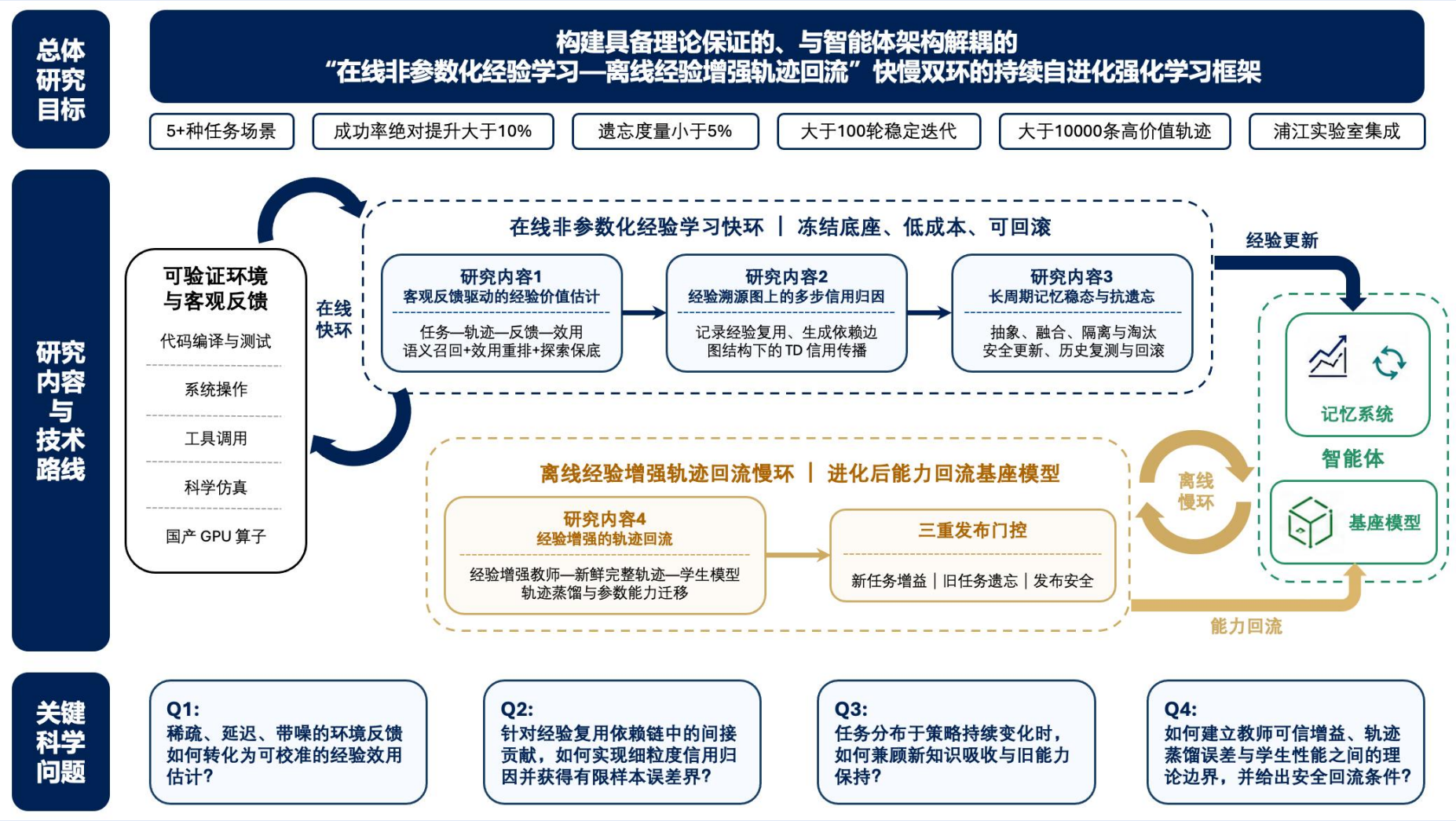
$$\Delta Q(m) \mathrel{+}= \alpha \sum_{m_0 \in A_j} (\gamma\lambda)^{d(m, m_0)} \cdot \delta(m_0, j)$$

- EC-MDP：分离外生任务流与内生 Memory 状态
- $d(m, m_0)$：经验依赖的最短图距离，而非时间间隔
- γ 控制结构视野；λ 调节深层归因的偏差—方差折中

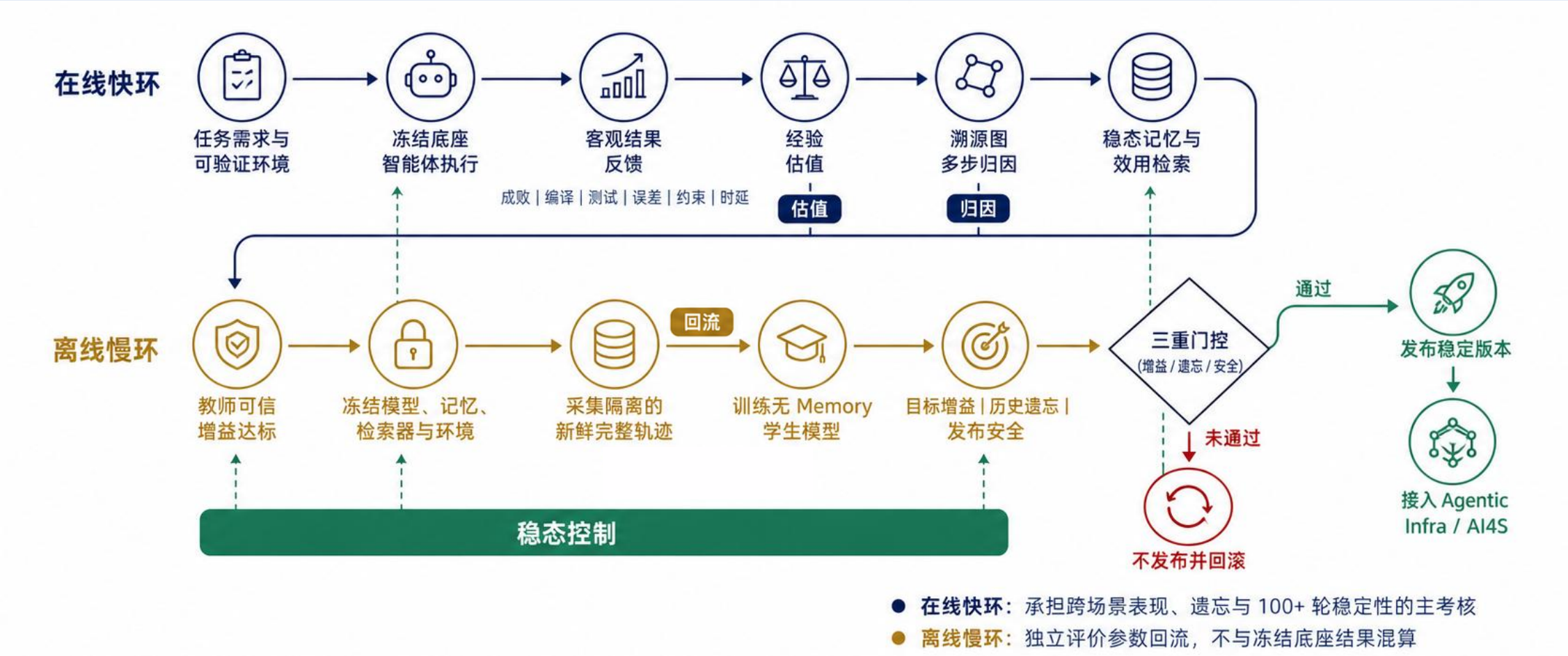
最大深度 | 折扣阀值 | 访问次数归一化 | 裁剪更新

时间顺序不等于因果来源，归因应沿经验依赖结构传播。

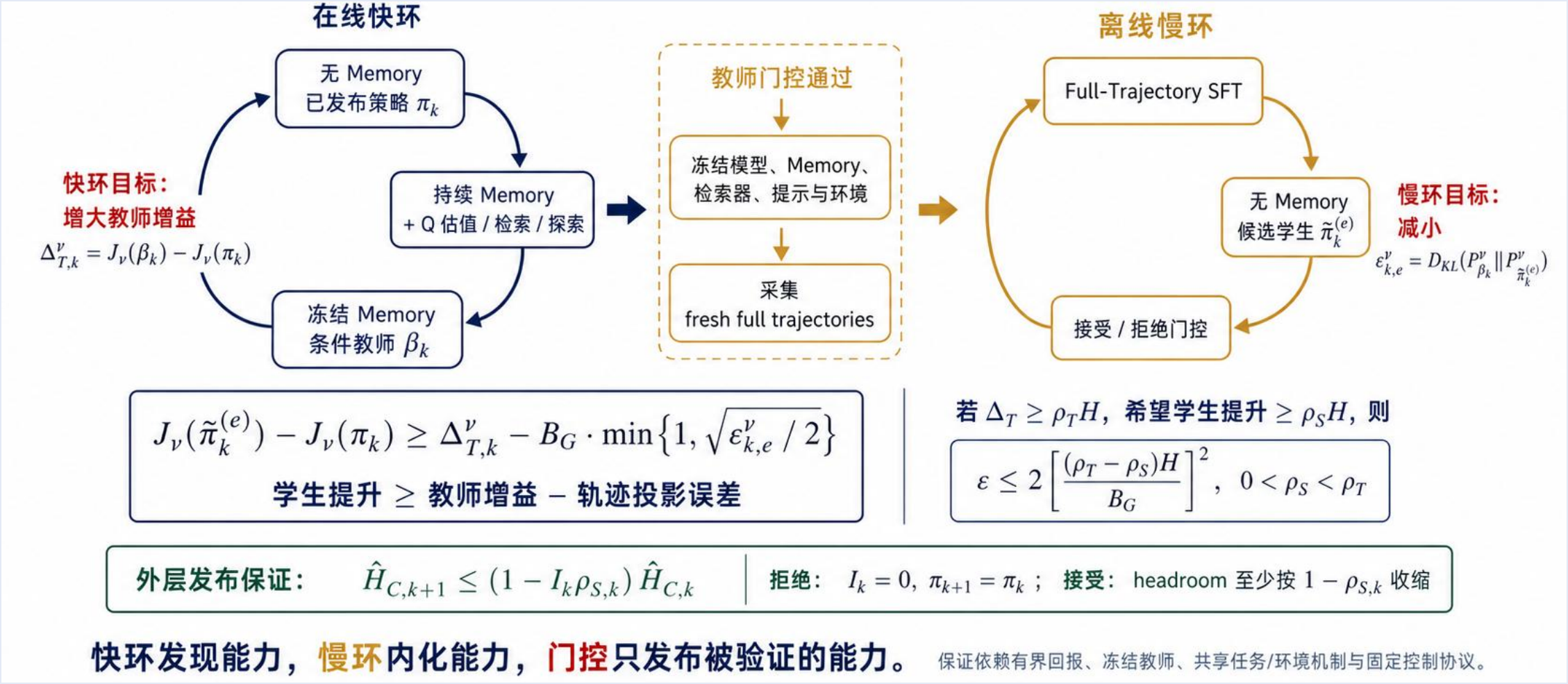
MemDist：在线进化与离线蒸馏的结合



经由快慢双环，经验从现场反馈持续转化为模型能力



教师轨迹回流：快环提升教师增益，慢环压缩投影误差



QCon
全球软件开发大会

InfoQ 技术大会

目录

1 AI 落地的“最后一公里”

2 基于记忆的智能体自进化

3 **垂域冷启动：NPU算子生成**

国产算子生成的AI化已箭在弦上，不得不发

痛点一

生态困境：国产GPU如昇腾系列硬件性能国际领先，但算子生态仍是发展瓶颈。如何打破“CUDA 壁垒”，完善算力主权迫在眉睫。

现状

1. **作坊式开发： 高度依赖稀缺的人类专家；**
2. **开发效率低： 即便是人类专家，单个高性能算子优化需耗时数周/月；**
3. **迭代速度差： 硬件发展速度飞快，传统算子开发方式难以跟上硬件与算法的快速更迭。**

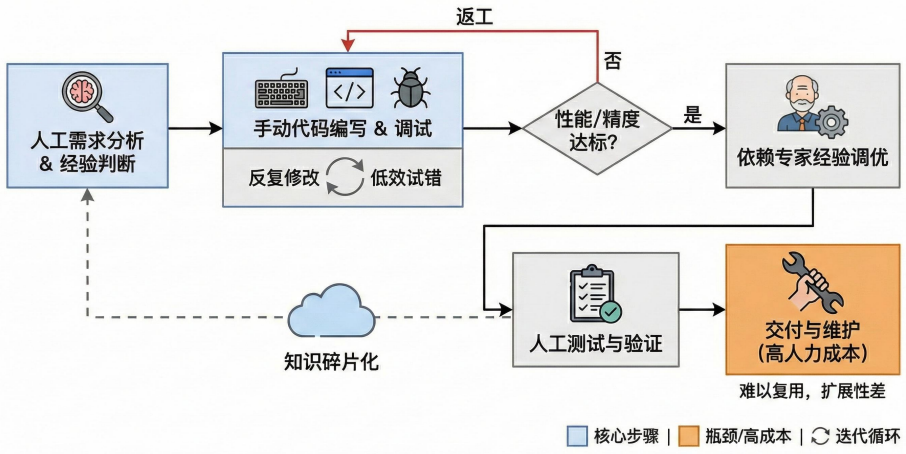


图1：传统作坊式算子开发流程示意

痛点二

现有 AI 方案的局限性：谋求通过自动化的 AI 取代手工作坊，但传统的训练垂类模型范式无法满足算子持续迭代需求。

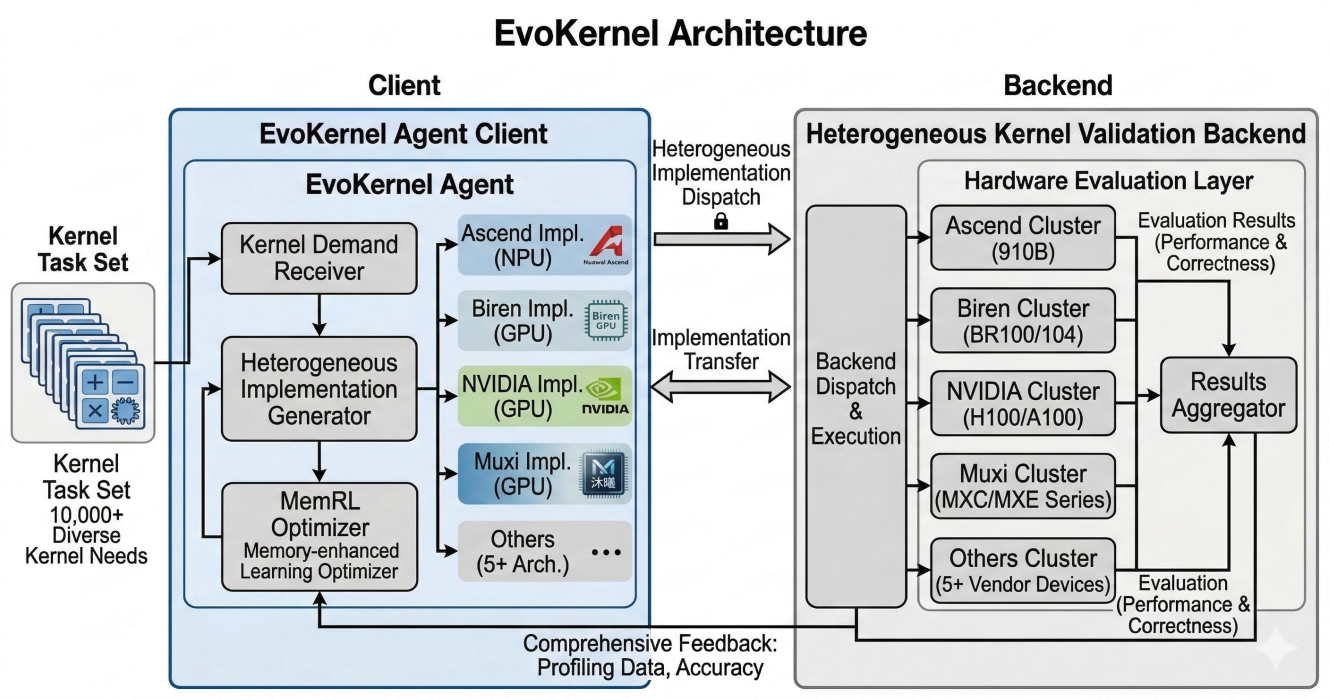
现状

1. **成本高昂： 算法、硬件架构更新为适配新算子、新架构频繁重训大模型不可行；**
2. **语料稀缺： 国产生态相关预料严重匮乏，亟须合理的自举机制；**
3. **约束复杂：算子约束复杂，Hack案件频发。**

Model	Level	CUDA (%)	Ascend C (%)
GPT-5.2	L1	92.0	14.0
	L2	90.0	2.0
DeepSeek-V3.2	L1	50.0	8.0
	L2	9.0	0.0
Qwen3-Coder-30B	L1	46.0	7.0
	L2	10.0	0.0

EvoKernel：记忆自进化 for NPU 算子开发

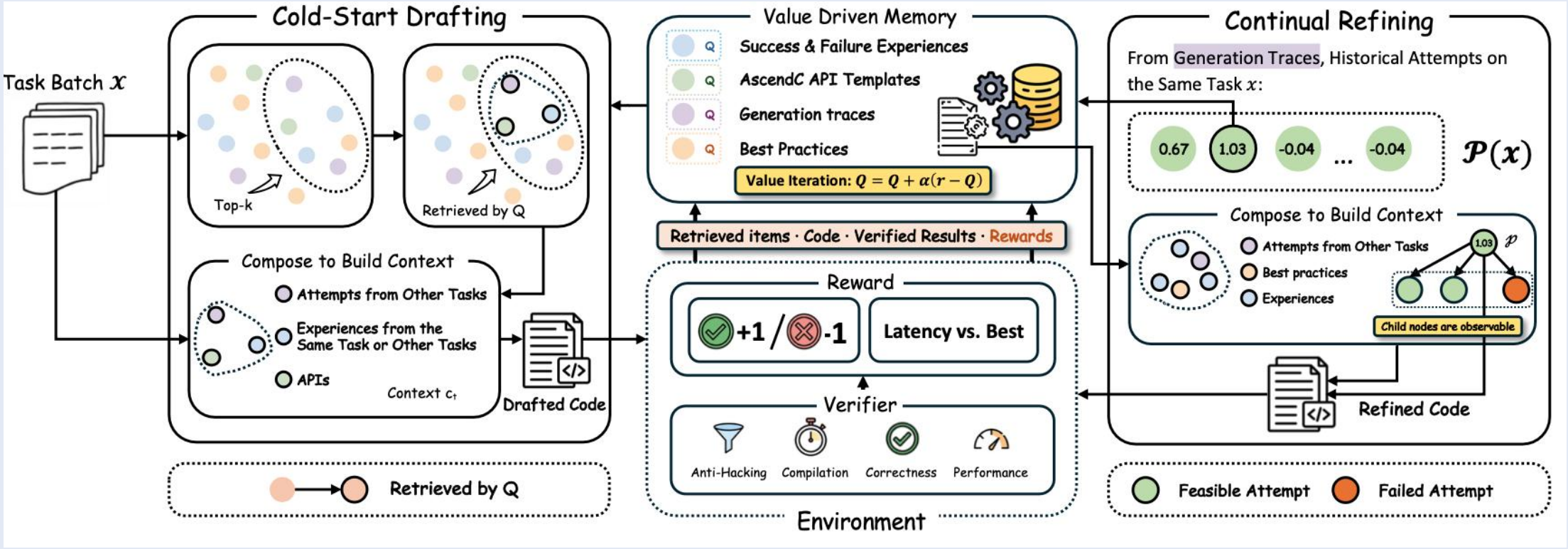
本项目的主要研发内容为构建一个名为 EvoKernel 的**算子开发智能体框架**，目前已迭代至2.0版本。
旨在为国产 GPU 生态提供一个能自动、低成本、非破坏性地持续学习和积累**算子开发**经验的**“AI 专家系统”**，
改变“作坊式”开发模式，长久地解决算力生态在硬件与算法飞速迭代背景下的“开发速度”瓶颈问题。



把算子开发从“手工作坊”改造成一条自动迭代流水线：

1. 任务输入
2. 冷启动草拟
3. 硬件验证
4. 记忆强化
5. 持续精炼

EvoKernel 框架



https://arxiv.org/abs/2601.03192

实际应用重点：谨防 Reward Hacking！

自进化最怕奖励不可信；算子场景的优势是奖励可以被机器验证；只要验证链足够严格，智能体每一次试错都能变成可靠训练信号，而不是自说自话。

Anti-hacking

先防作弊：不能绕过真实 NPU kernel 去假装成功

Compilation

再看能否编译：工程必须真的过工具链

Correctness

然后看数值：输出必须和参考实现一致

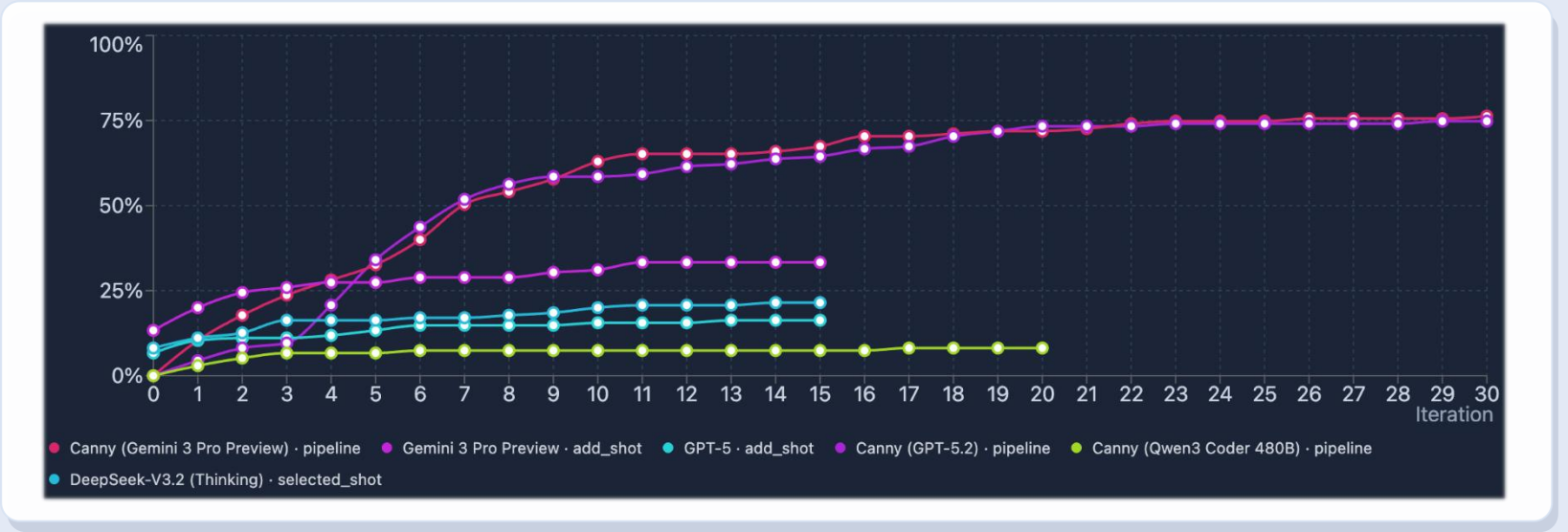
Performance

最后看速度：profiling latency 给出优化方向

有了可信奖励，记忆才不是玄学：
每条经验为什么被强化、为什么被降权，都可以追溯。

一次写对 v.s. 持续进化

记忆自进化的目的不是第一次就写对，而是系统能从错误里持续变好；正确率、提速和可优化算子数量共同说明：闭环不是在堆 prompt，而是在积累可复用的工程经验。



11% → 97%

NPU KernelBench 主实验正确率提升

4.27×

相对首次可行候选的中位提速

250+ ops

至少存在有效优化候选的算子集合

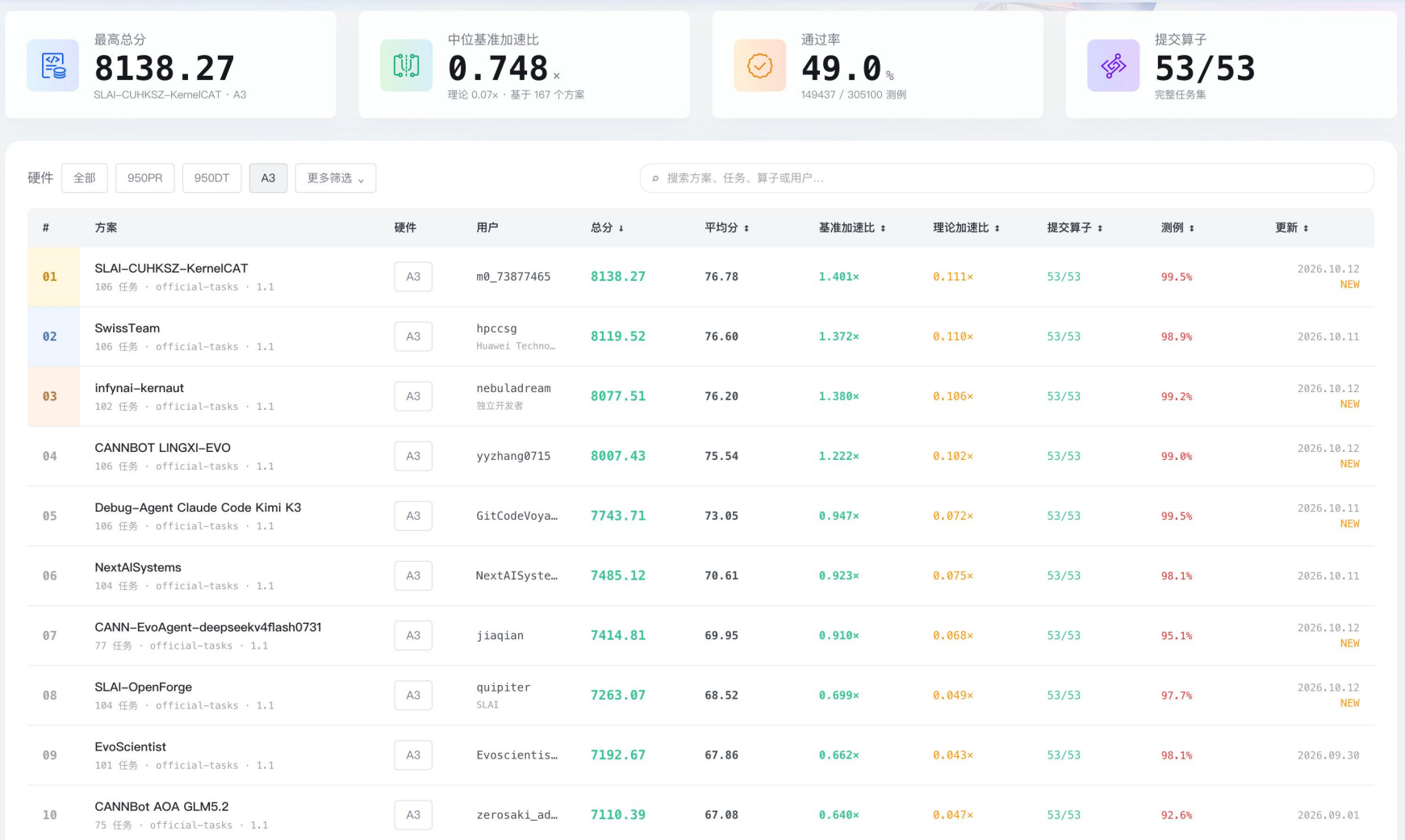
EvoKernel 结果：华为官方 CANN Bench

53/53

算子实现提交覆盖
L1—L4任务

第7名

A3榜单历史排名
得分：7414
2026年10月12日



#	方案	硬件	用户	总分	平均分	基准加速比	理论加速比	提交算子	测例	更新
01	SLAI-CUHKSZ-KernelCAT 106 任务 · official-tasks · 1.1	A3	m0_73877465	8138.27	76.78	1.401×	0.111×	53/53	99.5%	2026.10.12 NEW
02	SwissTeam 106 任务 · official-tasks · 1.1	A3	hpccsg Huawei Techno…	8119.52	76.60	1.372×	0.110×	53/53	98.9%	2026.10.11
03	infynai-kernaut 102 任务 · official-tasks · 1.1	A3	nebuladream 独立开发者	8077.51	76.20	1.380×	0.106×	53/53	99.2%	2026.10.12 NEW
04	CANNBOT LINGXI-EVO 106 任务 · official-tasks · 1.1	A3	yyzhang0715	8007.43	75.54	1.222×	0.102×	53/53	99.0%	2026.10.12 NEW
05	Debug-Agent Claude Code Kimi K3 106 任务 · official-tasks · 1.1	A3	GitCodeVoya…	7743.71	73.05	0.947×	0.072×	53/53	99.5%	2026.10.11 NEW
06	NextAISystems 104 任务 · official-tasks · 1.1	A3	NextAISyste…	7485.12	70.61	0.923×	0.075×	53/53	98.1%	2026.10.11
07	CANN-EvoAgent-deepseekv4flash0731 77 任务 · official-tasks · 1.1	A3	jiaqian	7414.81	69.95	0.910×	0.068×	53/53	95.1%	2026.10.12 NEW
08	SLAI-OpenForge 104 任务 · official-tasks · 1.1	A3	quipiter SLAI	7263.07	68.52	0.699×	0.049×	53/53	97.7%	2026.10.12 NEW
09	EvoScientist 101 任务 · official-tasks · 1.1	A3	Evoscientis…	7192.67	67.86	0.662×	0.043×	53/53	98.1%	2026.09.30
10	CANNBot AOA GLM5.2 75 任务 · official-tasks · 1.1	A3	zerosaki_ad…	7110.39	67.08	0.640×	0.047×	53/53	92.6%	2026.09.01

EvoKernel 结果：任务数量与数据飞轮on CANN

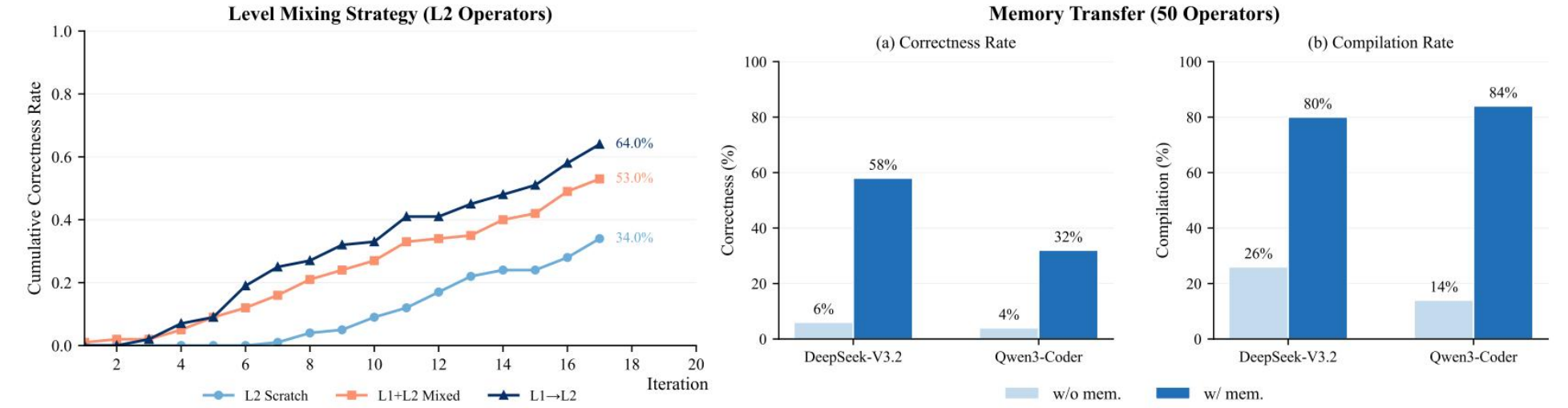
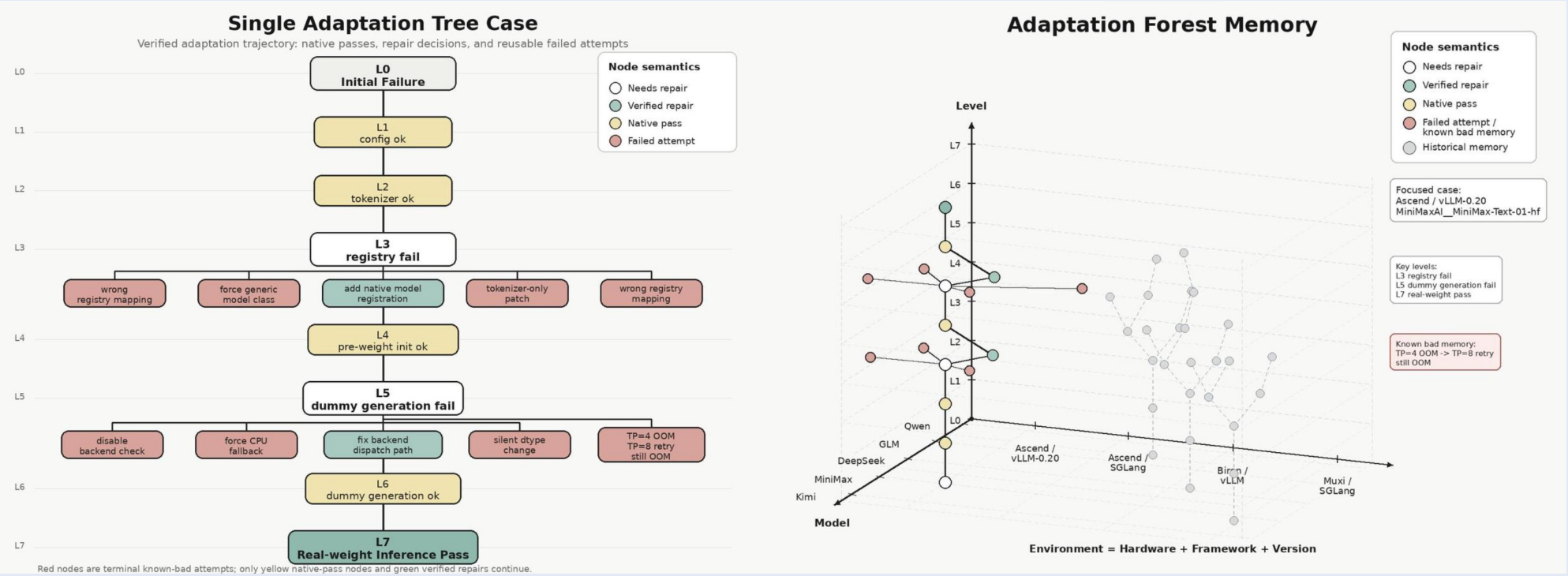


Figure 3: Transfer and generalization. **(Left)** Transfer across difficulty levels: cumulative success rate on L2 under different stream compositions. **(Right)** Transfer across generator backbones: performance on held-out operators when reusing memory built with GPT-5.2.

KernelX：模型级整网适配与优化

让 Agent 直接完成目标模型的端到端部署与吞吐量优化，用户不再需要感知底层硬件，期待彻底抹平算力生态差异。



AI Agent 驱动的模型整网适配优化

DeepSeek-V4 / GLM-5.1 / Qwen3.5 真实端到端落地

约 12 小时跑通

2.12× 端到端提速

Qwen3.5-9B，昇腾 910B2 混合架构
24 linear + 8 full，端到端快速跑通。

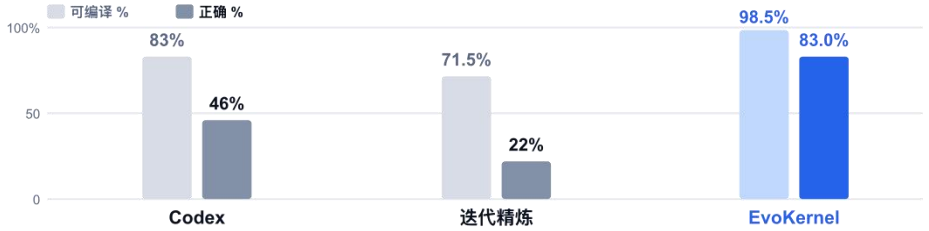
模型	平台	跑通时长 · 提速	状态
Qwen3.5-9B	SGLang	约 12h · 2.12×	接近可服务
DeepSeek-V4-Flash	SGLang	约 24h · 1.74×	端到端连贯
DeepSeek-V4-Pro 1.5T	SGLang	FP8 + FP4 连贯	serving 临界
GLM-5.1	SGLang	约 18h · 16×	端到端连贯
Qwen3.5 4B / 27B / 35B	SGLang	端到端已落地	已落地

同一模型不重训： 国产算子正确率 4.0% → 83.0%，可编译 11% → 98.5%（AscendC · GPT-5.2 · 30 轮）。

自动生成算子逼近厂商手写库，并已在四个国产平台真机连续验证。

B1 · 同一模型不重训 · 30 轮

国产算子自我进化（AscendC · GPT-5.2）



正确 = 可编译 ∧ PyTorch 一致 ∧ 真机达标；Codex 的 83% 为可编译率，非正确率。

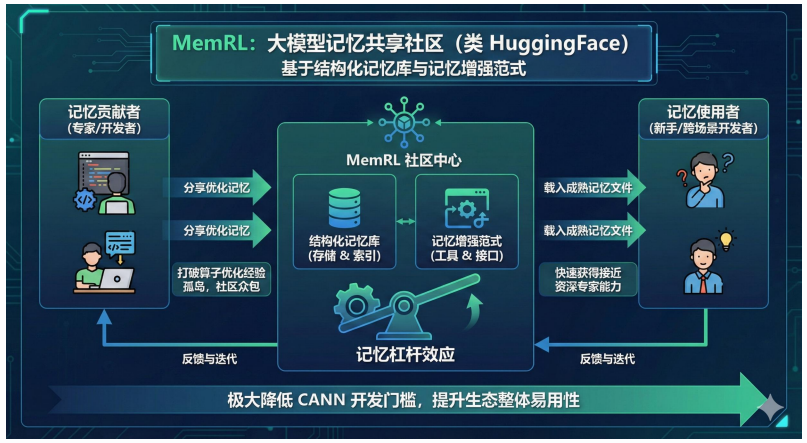
B2 · 逼近厂商手写库

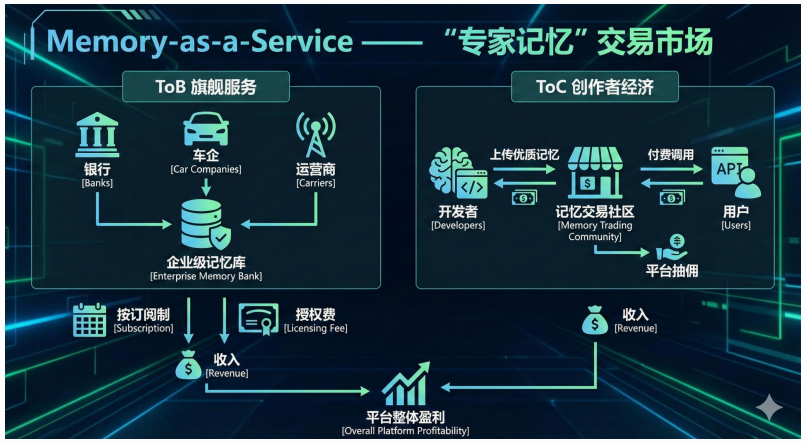
逼近厂商手写库（昇腾）



当静态数据枯竭，由 AI 产生的经验成为 AI 下半场新燃料

工程落地的终点不是更多 demo，而是一套会增长的经验资产层。当经验可以被验证、版本化、共享，它就不再是某个人的 Prompt 技巧，而是整个生态可以复用的基础设施。



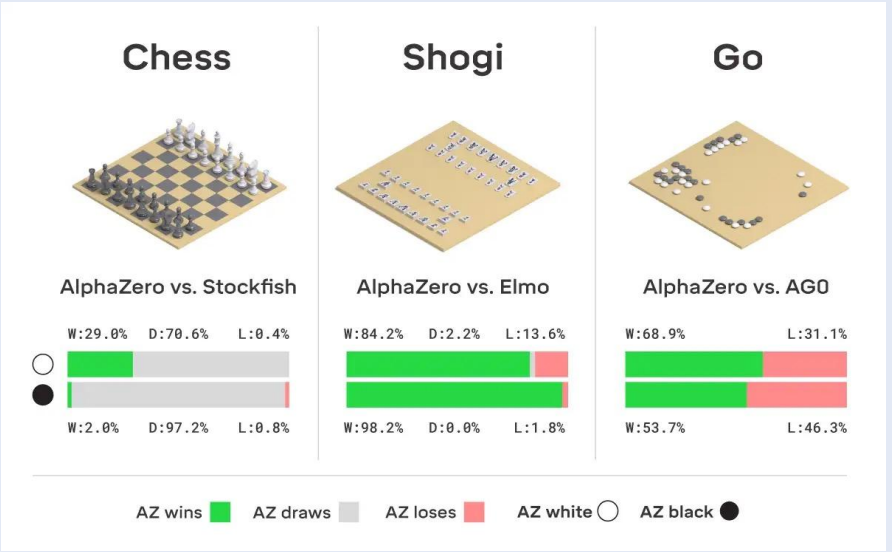


1 先找像的 → 2 再挑值的 → 3 现场验证 → 4 更新价值

当 AI 产生的经验超过人类，人类的经验将成为噪声？



AlphaGo, 2016



AlphaZero, 2017

其实才三年半


Sam Altman
@sama
today we launched ChatGPT. try talking with it here:
chat.openai.com
7:38 PM · Nov 30, 2022
小红书
小红书号：6209072797

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANK YOU

